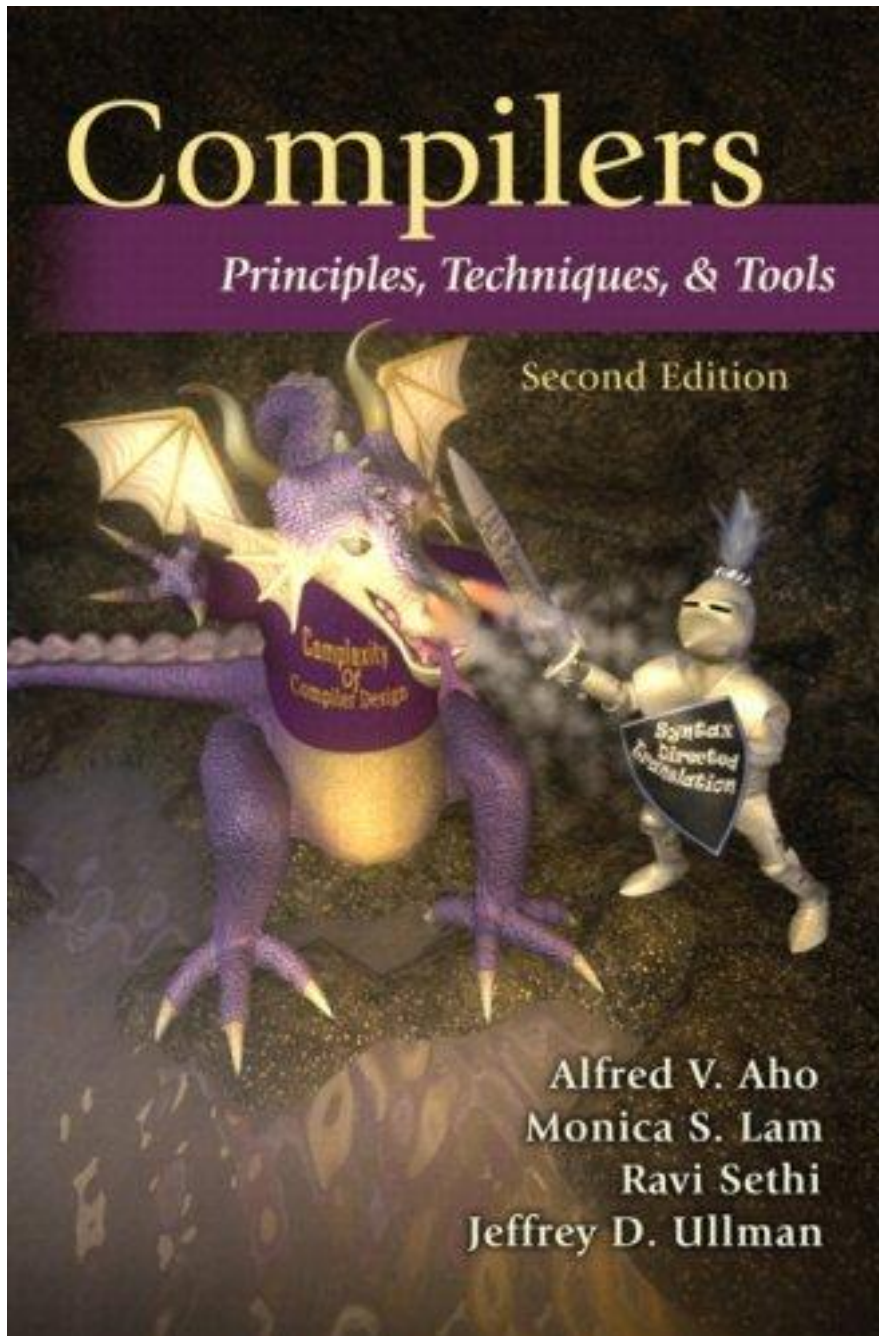




四川大學
SICHUAN UNIVERSITY

Principle of Compiler

luoyining@scu.edu.cn



Chapter 6

Intermediate-Code Generation

6.3 Types and Declarations

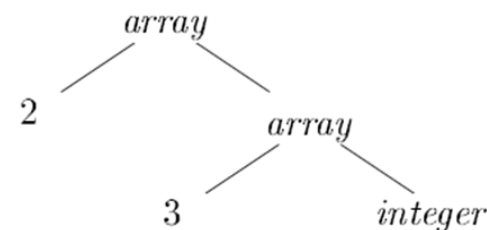
- 类型的应用分为两类
 - 类型检查：利用一组逻辑规则来检查运算分量的类型和运算符的预期类型是否匹配。
 - 翻译：根据类型信息确定存储空间、计算数组元素的地址、进行类型转换，等等。

6.3.1 Type Expressions

- 基本类型：int, float, char, boolean
- 把类型构造算子 (type constructor) 作用于类型表达式得到新的类型表达式，如数组、指针等。
- 为类型表达式命名 (typedef) ， 类型名也是类型表达式。
- array ： 作用于一个数字和一个类型表达式。
 - 若 T 是类型表达式， n是一个整数， 则array(n, T)是类型表达式。

• Example 6.8

Type expressions	Type	Variable
array(3, int)	int [3]	int a[3]
array(2, array(3, int))	int [2][3]	int b[2][3]



- 指针 pointer: 作用于类型表达式。
 - 若T 是类型表达式， 则 pointer(T) 是类型表达式。

pointer(int)	int *	int *p
array(3, pointer(int))	int (*)[3]	int (*)p[3]

type constructor

- 笛卡尔积 $\times$

- 若 T_1 和 T_2 是类型表达式, 则笛卡尔积 $T_1 \times T_2$ 是类型表达式。
- 引入笛卡尔积主要是为了保证定义的完整性。例如, 用于表示函数参数。

<code>int max(int, int)</code>	<code>int $\times$ int</code>
--------------------------------	--

- 记录 record : 作用于字段名和相应的类型表达式。

- 若有标识符 $N_1, N_2, \dots, N_n$ 与类型表达式 $T_1, T_2, \dots, T_n$, 则 `record (( $N_1 \times T_1$ )  $\times$  ( $N_2 \times T_2$ )  $\times$  ...  $\times$  ( $N_n \times T_n$ ))` 是一个类型表达式。

<code>record ((x$\times$int) $\times$ (y$\times$int))</code>	<code>struct point{ int x; int y; }</code>	<code>struct point cen;</code>
<code>record ((x$\times$int) $\times$ (y$\times$int) $\times$ (r$\times$double))</code>	<code>struct circle{ int x; int y; double r; }</code>	<code>struct circle cir;</code>

- 函数构造算子 $\rightarrow$

- 若 $T_1, T_2, \dots, T_n$ 和 R 是类型表达式, 则 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 是类型表达式。

<code>int max(int, int)</code>	<code>int $\times$ int $\rightarrow$ int</code>
<code>char* fun(char*, char*)</code>	<code>pointer(char) $\times$ pointer(char) $\rightarrow$ pointer(char)</code>

6.3.2 Type Equivalence

- 不同的语言有不同的类型等价的定义。类型等价是类型系统（6.5 类型检查）的一个组成部分。
- 利用类型表达式进行类型等价判断
 - 结构等价
 - 名字等价

```
struct S1 { int a; };  
struct S2 { int a; };  
  
struct S1 x;  
struct S2 y;  
x.a = 1;  
y.a = 1;  
  
if (x.a == y.a) printf("equal"); else printf("not equal");  
if (x == y) printf("equal"); else printf("not equal");
```

类型表达式的结构等价测试（不考虑环）

```
boolean sequiv( typeExpression s, typeExpression t )  
{  
    if  $s$  和  $t$  是相同的基本类型  
        return true;  
    else if  $s == array(s_1, s_2)$  and  $t == array(t_1, t_2)$   
        return sequiv( $s_1, t_1$ ) and sequiv( $s_2, t_2$ );  
    else if  $s == s_1 \times s_2$  and  $t == t_1 \times t_2$   
        return sequiv( $s_1, t_1$ ) and sequiv( $s_2, t_2$ );  
    else if  $s == pointer(s_1)$  and  $t == pointer(t_1)$   
        return sequiv( $s_1, t_1$ );  
    else if  $s == s_1 \rightarrow s_2$  and  $t == t_1 \rightarrow t_2$   
        return sequiv( $s_1, t_1$ ) and sequiv( $s_2, t_2$ );  
    else return false;  
}
```

—— 陈意云、张昱，《编译原理》（第3版），高等教育出版社

6.5.2 Type Conversions

- **Example**

$2 * 3.14 \longrightarrow \begin{array}{l} t_1 = (\text{float})2 \\ t_2 = t_1 * 3.14 \end{array}$

- 1. 整数和浮点数的表示形式不同
- 2. 整数运算和浮点数运算的机器指令不同

- Intel 8086处理器只能处理整数运算。独立浮点协处理器芯片8087与每一代处理器一起升级。
- 当Intel486出现时，浮点硬件就被集成到主CPU中，称为FPU。

- SDD/SDT for type conversions

$E \rightarrow E_1 + E_2$	<pre>E.addr = new Temp() E.code = E₁.code E₂.code gen(E.addr '=' E₁.addr '+' E₂.addr) if (E₁.type = integer and E₂.type = integer) E.type = integer; else if (E₁.type = float and E₂.type = integer) ...</pre>
---------------------------	---

```
E → E1 + E2 { E.type = max(E1.type, E2.type);  
a1 = widen(E1.addr, E1.type, E.type);  
a2 = widen(E2.addr, E2.type, E.type);  
E.addr = new Temp();  
gen(E.addr '=' a1 '+' a2); }
```

Figure 6.27 Introducing type conversions into expression evaluation

Example

Production	Semantic Rules
$E \rightarrow E_1 / T$	$E.type = (E_1.type == \mathbf{int} \text{ and } T.type == \mathbf{int} ? \mathbf{int} : \mathbf{float})$ $E.val = (E.type == \mathbf{int} ? E_1.val \ \mathbf{div} \ T.val : E_1.val / T.val)$
$E \rightarrow T$	$E.type = T.type \quad E.val = T.val$
$T \rightarrow \mathbf{num}$	$T.type = \mathbf{int} \quad T.val = \mathbf{num}.val$
$T \rightarrow \mathbf{num.num}$	$T.type = \mathbf{float} \quad T.val = \mathbf{num}. \mathbf{num}.val$

Production	Semantic Rules (Lvalue and Rvalue)
$E \rightarrow E_1 = E_2$	if($E_1.val == r$) print("error"); $E.val = r$;
$E \rightarrow E_1 + E_2$	
$E \rightarrow E_1 ++$	if ($E_1.val == r$) print("error"); $E.val = r$;
$E \rightarrow (E_1)$	$E.val = E_1.val$
$E \rightarrow \mathbf{id}$	$E.val = l$;
$E \rightarrow \mathbf{num}$	$E.val = r$;

Example 4.48

$$\begin{array}{lcl}
 S & \rightarrow & L = R \mid R \\
 L & \rightarrow & *R \mid \mathbf{id} \\
 R & \rightarrow & L
 \end{array}$$

6.3.3 Declarations

- 类型表达式在变量声明语句中的作用：
 - 声明变量;
 - 计算变量在运行时刻需要的内存数量;
 - 计算变量在存储区域的相对地址。
- 变量的语义信息：
 - 类型,
 - 存储宽度,
 - 相对地址等
- 不同的变量声明文法

$$D \rightarrow T L$$

$$T \rightarrow \text{int} \mid \text{float}$$

$$L \rightarrow L, \text{id} \mid \text{id}$$

1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $addType(\text{id}.entry, L.inh)$
5) $L \rightarrow \text{id}$	$addType(\text{id}.entry, L.inh)$

int x,y,z;

$$D \rightarrow T \text{id} ; D \mid \varepsilon$$

$$T \rightarrow B C \mid \text{record} \{ ' D ' \}$$

$$B \rightarrow \text{int} \mid \text{float}$$

$$C \rightarrow \varepsilon \mid [\text{num}] C$$

- D 生成声明语句序列, 以分号结束
- T 生成不同的类型, 包括基本类型、数组和记录
- B 生成基本类型int或float
- C 决定 T 生成的是基本类型还是数组类型

int x;
int y;
int z;
int [2][3]a;

6.3.4 Storage Layout for Local Names

- 类型 (SDD)

• Example 5.13

- `int [2][3]`

$$T \rightarrow B C$$

$$B \rightarrow \text{int} \mid \text{float}$$

$$C \rightarrow \epsilon \mid [\text{num}] C$$

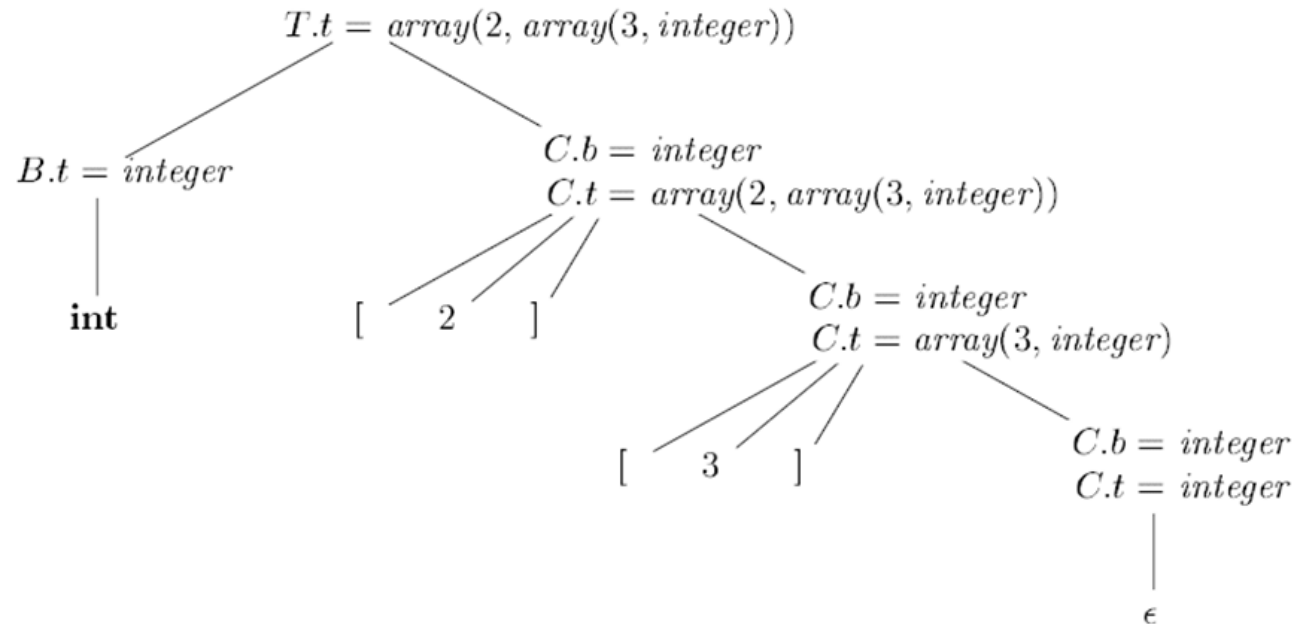


Figure 5.17: Syntax-directed translation of array types

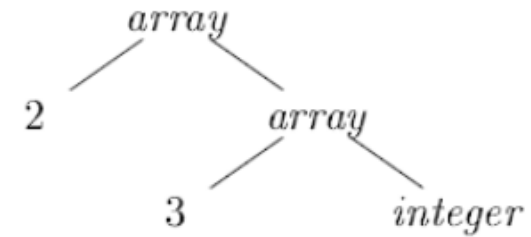


Figure 5.15: Type expression for `int[2][3]`

PRODUCTION	SEMANTIC RULES
$T \rightarrow B C$	$T.t = C.t$ $C.b = B.t$
$B \rightarrow \text{int}$	$B.t = \text{integer}$
$B \rightarrow \text{float}$	$B.t = \text{float}$
$C \rightarrow [\text{num}] C_1$	$C.t = \text{array}(\text{num.val}, C_1.t)$ $C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

Figure 5.16: T generates either a basic type or an array type

6.3.4 Storage Layout for Local Names

- 类型和存储宽度 (SDT)

$$T \rightarrow B \ C$$

- **Example 6.9**

$$B \rightarrow \mathbf{int} \mid \mathbf{float}$$
$$C \rightarrow \varepsilon \mid [\mathbf{num}] C$$

T	<i>type, width</i>
B	<i>type, width</i>
C	<i>type, width</i>

- `int [2][3]`

- How the type and width are passed from B, down the chain of C's ?

$T \rightarrow B$	② { $\textcolor{red}{t} = B.type; \textcolor{red}{w} = B.width; \}$
C	⑤ { $T.type = C.type; T.width = C.width; \}$
$B \rightarrow \text{int}$	① { $B.type = int; B.width = 4; \}$
$B \rightarrow \text{float}$	① { $B.type = float; B.width = 8; \}$
$C \rightarrow \varepsilon$	③ { $C.type = \textcolor{red}{t}; C.width = \textcolor{red}{w}; \}$
$C \rightarrow [\text{num}]C_1$	④ { $C.type = array(num.value, C_1.type);$ $C.width = num.value * C_1.width; \}$

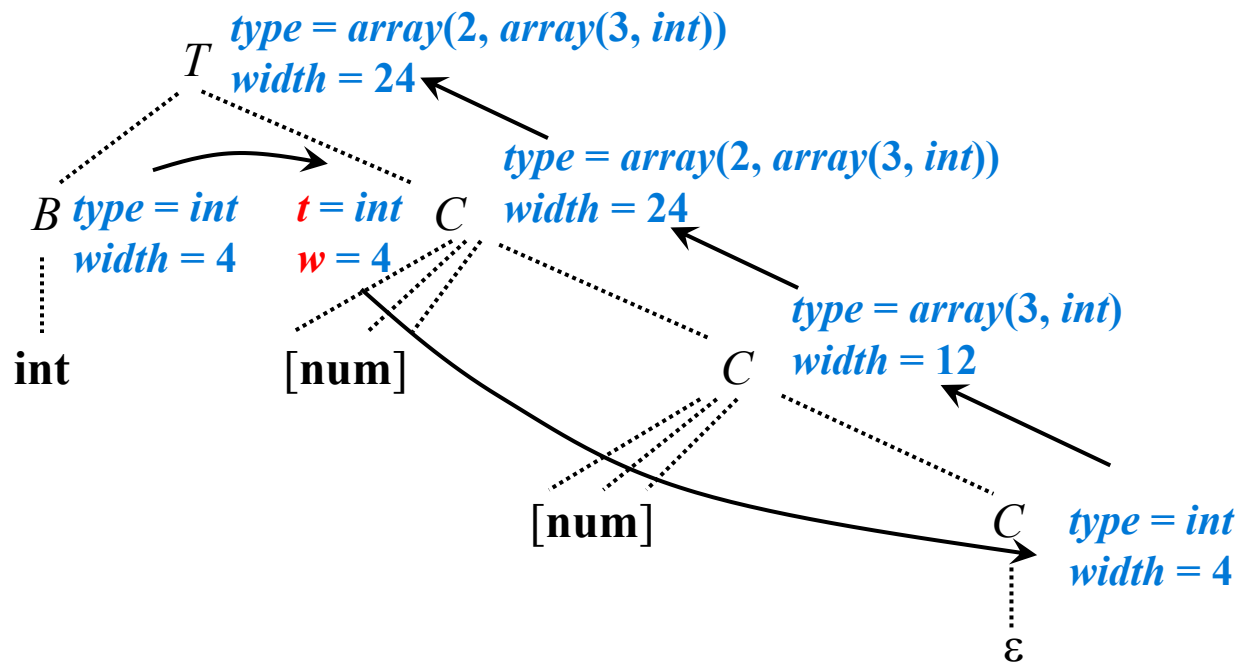


Figure 6.15: Computing types and their widths

Figure 6.16: Syntax-directed translation of array types

6.3.5 Sequences of Declarations

- 相对地址 (SDT)
 - 函数中连续定义的局部变量总是分配在连续的存储区间内, 因此给每个变量分配一个相对于这个区间开始位置的相对地址。

• `int [2][3]a;    int b;`

$P \rightarrow \{ \text{offset} = 0; \} D$

$D \rightarrow T \text{id}; \{ \text{top.put}(\text{id.lexeme}, T.\text{type}, \text{offset});$
 $\text{offset} = \text{offset} + T.\text{width}; \}$

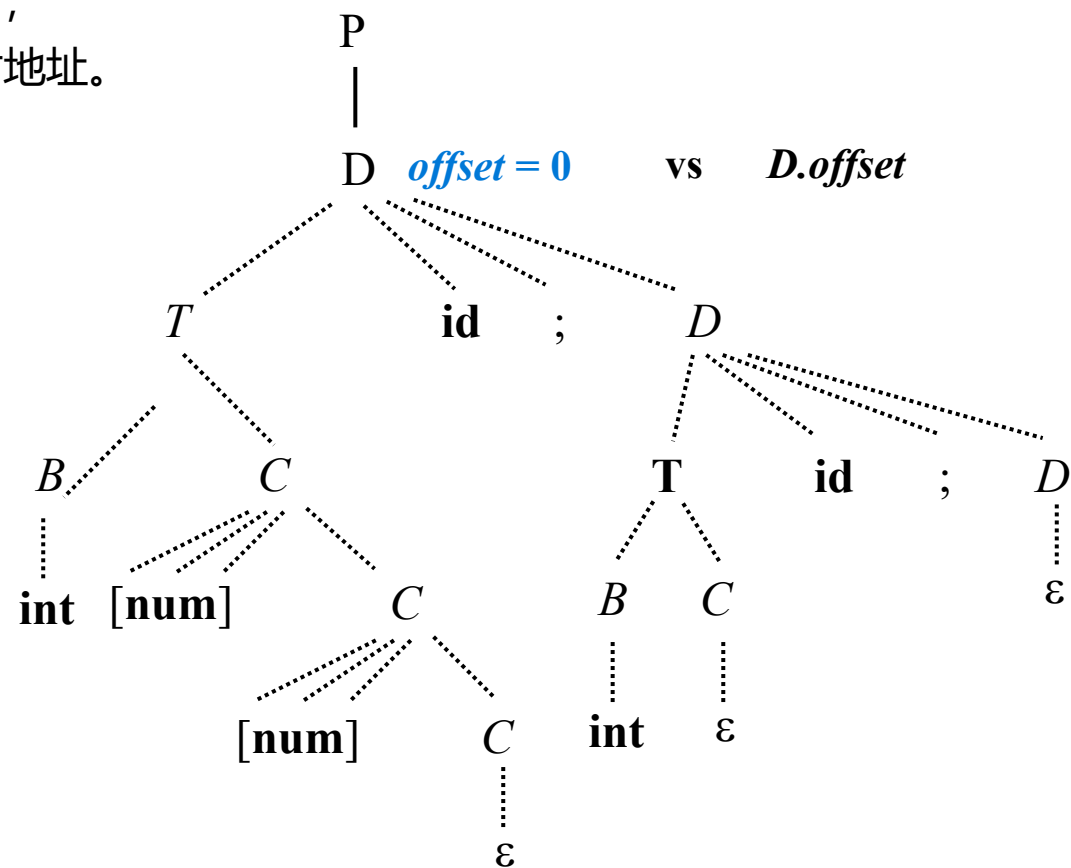
D_1

$D \rightarrow \varepsilon$

$T \rightarrow B C$

$B \rightarrow \text{int} \mid \text{float}$

$C \rightarrow \varepsilon \mid [\text{num}] C$



- top 指向当前符号表; 对每一个id: 添加到符号表中, 写入类型、相对地址信息

作用域

- 一个声明的作用域是指该声明起作用的那一部分程序（块结构）。
- Example 2.15 & Example 2.16

```
1)  {   int x1; int y1;  
2)    {   int w2; bool y2; int z2;  
3)        ... w2 ...; ... x1 ...; ... y2 ...; ... z2  
4)    }  
5)        ... w0 ...; ... x1 ...; ... y1 ...;  
6) }
```

- Three blocks: B_0 , B_1 , B_2

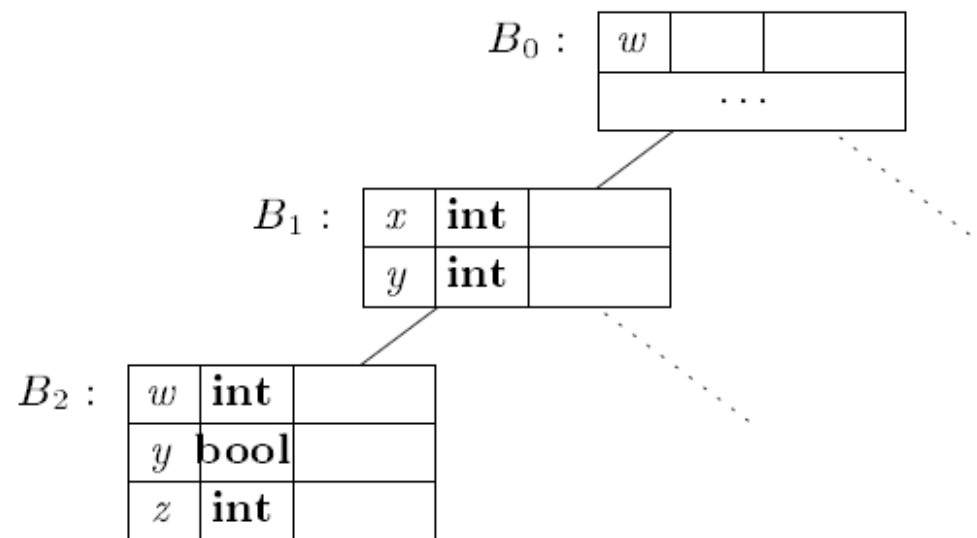


Figure 2.36: Chained symbol tables for Example 2.15

6.4 Translation of Expressions

6.4.1 Operations Within Expressions

- 翻译的主要任务：把表达式 / 赋值语句翻译成三地址码。
- 基本文法

$S \rightarrow \mathbf{id} = E;$

$E \rightarrow E_1 + E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow \mathbf{id}$

- SDD中属性的设计

V_N	Attributes
S	<i>code</i>
E	<i>code, addr</i>

<i>code</i>	$x = y \text{ op } z$	赋值指令
	$x = \text{op } y$	
	$x = y$	复制指令
<i>addr</i>	变量、常量或编译器生成的临时变量的地址	

SDD

- **Example 6.11**

$a = b + (-c); \longrightarrow \begin{array}{l} t_1 = \text{minus } c \\ t_2 = b + t_1 \\ a = t_2 \end{array}$

Production	Semantic Rules
$S \rightarrow \mathbf{id} = E;$	$S.\text{code} = E.\text{code} \parallel \text{gen}(\text{top.get}(\text{id.lexeme}) \text{'=' } E.\text{addr})$
$E \rightarrow E_1 + E_2$	$E.\text{addr} = \text{new Temp}()$ $E.\text{code} = E1.\text{code} \parallel E2.\text{code} \parallel \text{gen}(E.\text{addr} \text{'=' } E1.\text{addr} \text{'+' } E2.\text{addr})$
$E \rightarrow -E_1$	$E.\text{addr} = \text{new Temp}()$ //new Temp()生成一个临时变量, 并返回该变量地址 $E.\text{code} = E1.\text{code} \parallel \text{gen}(E.\text{addr} \text{'=' 'minus' } E1.\text{addr})$
$E \rightarrow (E_1)$	$E.\text{addr} = E1.\text{addr}$ $E.\text{code} = E1.\text{code}$
$E \rightarrow \mathbf{id}$	$E.\text{addr} = \text{top.get}(\text{id.lexeme})$ // top代表当前符号表 $E.\text{code} = ''$

SDT

PRODUCTION	SEMANTIC RULES
$S \rightarrow \text{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(\text{id.lexeme}) '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr '=' E_1.addr '+' E_2.addr)$
$ - E_1$	$E.addr = \text{new Temp}()$ $E.code = E_1.code \parallel$ $gen(E.addr '=' \text{'minus'} E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \text{id}$	$E.addr = top.get(\text{id.lexeme})$ $E.code = ''$

Figure 6.19: Three-address code for expressions

$S \rightarrow \text{id} = E ; \{ \textcolor{blue}{p = lookup(id.lexeme)}; \textcolor{blue}{if } p == \textcolor{blue}{null} \textcolor{blue}{then error} ;$
 $S.code = E.code \parallel gen(p '=' E.addr); \}$

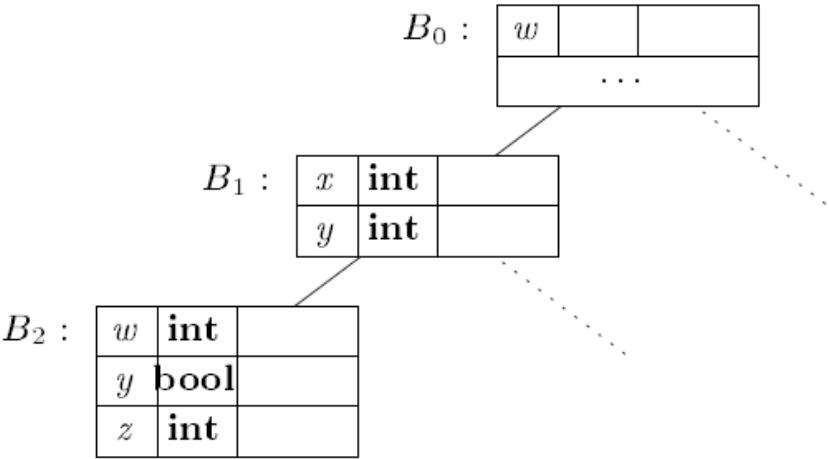


Figure 2.36: Chained symbol tables for Example 2.15

6.4.2 Incremental Translation

- code属性的计算过程就是一个不断地在原有三地址码的基础之上添加新的三地址码的过程。因此，可以用增量的方式生成code属性。

$$\begin{aligned} S &\rightarrow \text{id} = E ; \quad \{ \text{gen}(\text{top.get}(\text{id.lexeme}) '=' E.addr); \} \\ E &\rightarrow E_1 + E_2 \quad \{ E.addr = \text{new Temp}(); \\ &\quad \text{gen}(E.addr '=' E_1.addr '+' E_2.addr); \} \\ &| - E_1 \quad \{ E.addr = \text{new Temp}(); \\ &\quad \text{gen}(E.addr '=' \text{'minus'} E_1.addr); \} \\ &| (E_1) \quad \{ E.addr = E_1.addr; \} \\ &| \text{id} \quad \{ E.addr = \text{top.get}(\text{id.lexeme}); \} \end{aligned}$$

Figure 6.20: Generating three-address code for expressions incrementally

6.6 Control Flow

- if语句、while语句的翻译是和布尔表达式的翻译结合在一起的。

$$S \rightarrow \mathbf{if} (B) S_1 \mid \mathbf{if} (B) S_1 \mathbf{else} S_2 \mid \mathbf{while} (B) S_1$$

6.6.1 Boolean Expressions

- 基本文法

$$B \rightarrow B \parallel B \mid B \&\& B \mid !B \mid E \text{ rel } E \mid \text{true} \mid \text{false}$$

- 基本用途

- 改变控制流: `if (a < b && c < d)`

- 翻译成跳转指令

- 计算逻辑值: `x = a < b && c < d`

- 使用带有逻辑运算符的三地址指令进行求值

- 计算方法

- 程序设计语言的语义决定了是否需要对于一个布尔表达式的各个部分都进行求值;

- 如果不需要全部求值, 则编译器可以优化求值过程。

`1 or (not 0 and 0)`

`= 1 or (1 and 0)`

`= 1 or 0`

`= 1`

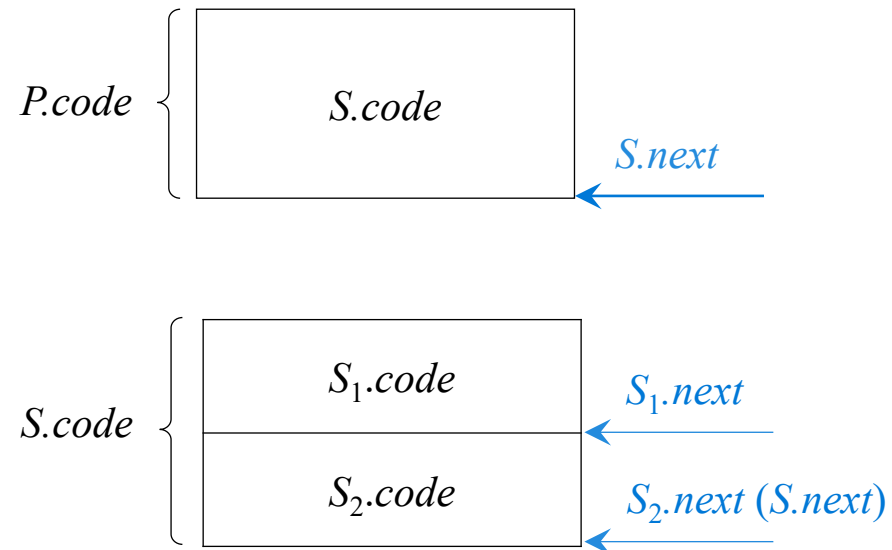
6.6.2 Short-Circuit Code

- 在短路代码（跳转代码）中，布尔运算符&&、||、!被翻译成跳转指令，运算符本身不会出现在代码中。

	三地址码/三地址指令	汇编指令
无条件转移	<code>goto L</code>	<code>jmp L</code>
条件转移	<code>if x goto L</code>	<code>cmp x, 0</code> <code>jne L</code>
	<code>if x relop y goto L</code>	<code>cmp x, y</code> <code>j** L</code>

6.6.3 Flow-of-Control Statements

- 基本文法

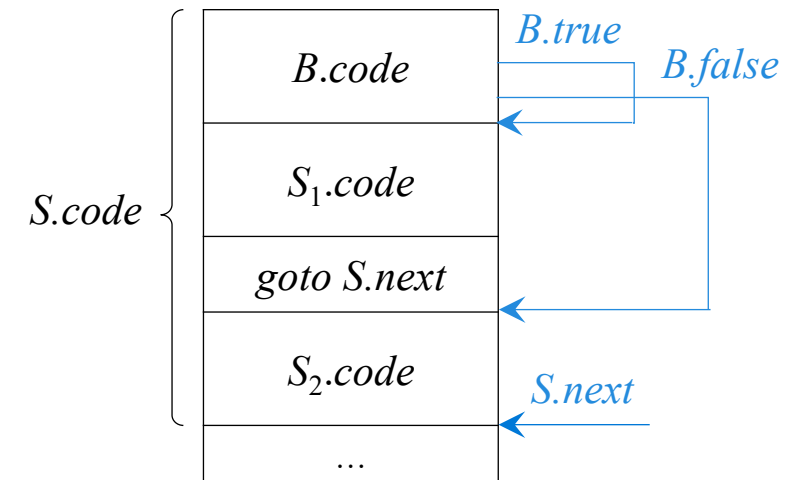
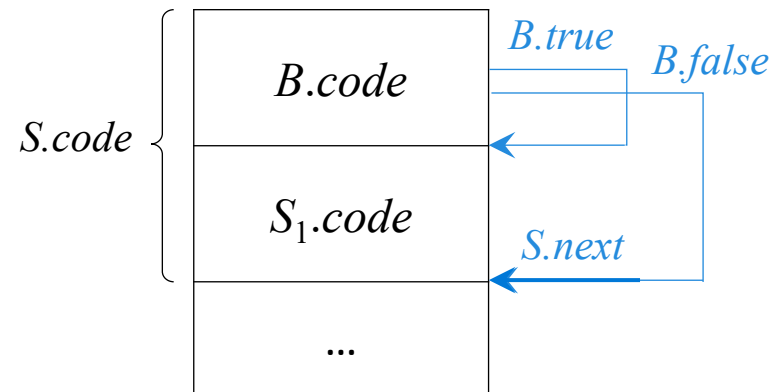
$$P \rightarrow S$$
$$S \rightarrow S_1 S_2$$
$$S \rightarrow \text{assign} \mid \text{if}(B) S_1 \mid \text{if}(B) S_1 \text{ else } S_2 \mid \text{while}(B) S_1$$


Production	Semantic Rules
$P \rightarrow S$	$S.next = \text{newlabel}();$ $P.code = S.code \parallel \text{label}(S.next)$
$S \rightarrow S_1 S_2$	$S_1.next = \text{newlabel}();$ $S_2.next = S.next$ $S.code = S_1.code \parallel \text{label}(S_1.next) \parallel S_2.code$
$S \rightarrow \text{assign}$	$S.code = \text{assign.code}$

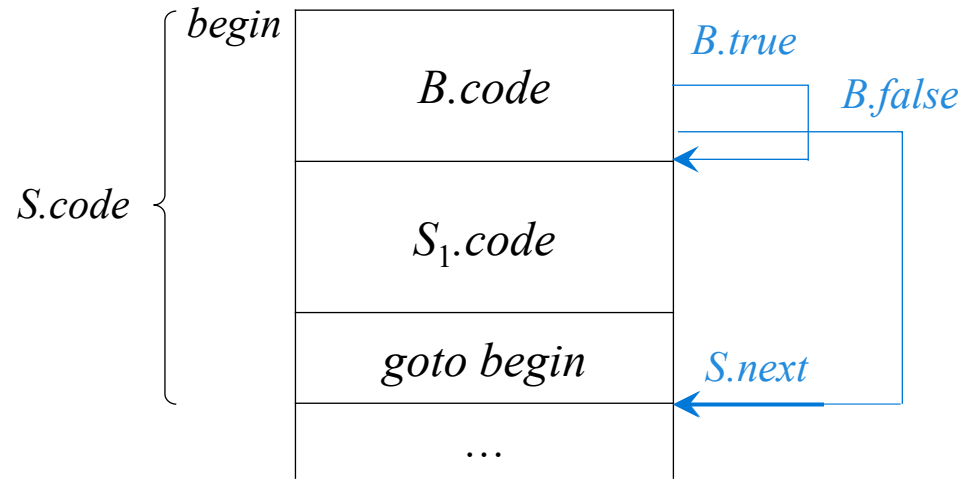
Code layout for if-, if-else-statements

Production	Semantic Rules
$S \rightarrow \text{if } (B) S_1$	$B.\text{true} = \text{newlabel}()$ $B.\text{false} = S.\text{next}()$ $S.\text{code} = B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$B.\text{true} = \text{newlabel}()$ $B.\text{false} = \text{newlabel}()$ $S.\text{code} = B.\text{code} \parallel \text{label}(B.\text{true}) \parallel S_1.\text{code} \parallel$ $\quad \text{gen('goto' } S.\text{next}) \parallel \text{label}(B.\text{false}) \parallel S_2.\text{code}$

- $B : \text{code}, \text{true}, \text{false}$
- $S : \text{code}, \text{next}$



Code layout for while-statements



Production	Semantic Rules
$S \rightarrow \mathbf{while} (B) S_1$	<pre>begin = newlabel() B.true = newlabel() B.false = S.next() S.code = label(begin) B.code label(B.true) S1.code gen('goto' begin)</pre>

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if} (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' S.next)$ $\quad \parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

- **Example 6.5** `do i=i+1; while (a[i]<v);`

```

L2:  t1 = i + 1
      i = t1
      t2 = i * 8
      t3 = a [ t2 ]
      if t3 < v goto L2

```

L₁:

- `while (a[i]<v) i=i+1;`
- `if (x<100) x = x+1; else x = x-1;`
- `if ( x>200 && x!=y ) x = x+2;`

Figure 6.36: Syntax-directed definition for flow-of-control statements.

- while (a[i]<v) i=i+1;

```
begin: t1 = i * 8
      t2 = a[t1]
      if t2 < v goto L1
      goto L2
L1: t3 = i + 1
    i = t3
    goto begin
L2: .....
```

- if (x>200 && x!=y) x = x+2;

```
      if x > 200 goto L1:
      goto L2
L1: if x != y goto L3
L3: t1 = x + 2
    x = t1
L2:.....
```

- if (x<100) x = x+1; else x = x-1;

```
      if x < 100 goto L1
      goto L2
L1: t1 = x + 1
    x1 = t1
    goto L3
L2: t2 = x - 1
    x2 = t2
L3: x =  $\varphi$ (x1, x2)
    .....
```

6.6.4 Control-Flow Translation of Boolean Expressions

- 基本文法

$$B \rightarrow B \parallel B$$
$$| B \ \&\& \ B$$
$$| !B$$
$$| E \ \text{rel} \ E$$
$$| \text{true}$$
$$| \text{false}$$

优先级: $! > \&\& > \parallel$

rel: $<, <=, =, !=, >, \text{or} >=$

$$B \rightarrow B \parallel C \mid C$$
$$C \rightarrow C \ \&\& \ D \mid D$$
$$D \rightarrow !D \mid E$$
$$E \rightarrow E \ \text{rel} \ E \mid F$$
$$F \rightarrow \text{true} \mid \text{false}$$

// 最低优先级: $\parallel$

// 中间优先级: $\&\&$

// 最高优先级: $!$ 和原子

// 关系运算 (如果存在)

// 原子

- 翻译任务: 把布尔表达式翻译成三地址码。

B	$code, \ true, \ false$	goto L	无条件转移指令
E	$code, \ addr$	if x goto L	条件转移指令
rel	op	if x relop y goto L	

SDD – I

$$\begin{aligned}
 B \rightarrow & B \parallel B \\
 & | B \ \&\& \ B \\
 & | !B \\
 & | E \ \mathbf{rel} \ E \\
 & | \mathbf{true} \\
 & | \mathbf{false}
 \end{aligned}$$

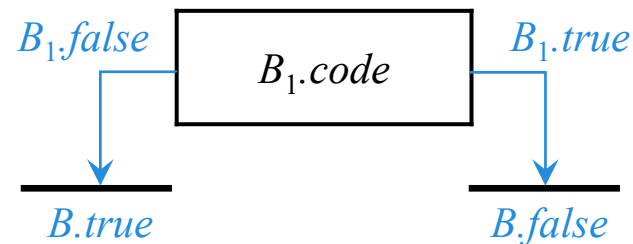
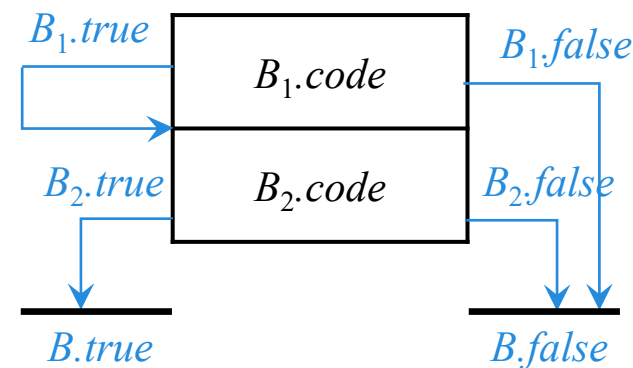
Production	Semantic Rules	Instructions	
$B \rightarrow E_1 \ \mathbf{rel} \ E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel gen('if' \ E_1.addr \ \mathbf{rel.op} \ E_2.addr \ 'goto' \ B.true)$ $\parallel gen('goto' \ B.false)$	$\mathbf{if} \ E_1.addr \ \mathbf{rel} \ E_2.addr \ \mathbf{goto} \ B.true$ $\mathbf{goto} \ B.false$ //根据比较运算的结果跳转到相应的位置, // 翻译成条件转移指令。	
$B \rightarrow \mathbf{true}$	$B.code = gen('goto' \ B.true)$	$\mathbf{goto} \ B.true$	// B 的值是常量, 翻译成无条件转移指令。
$B \rightarrow \mathbf{false}$	$B.code = gen('goto' \ B.false)$	$\mathbf{goto} \ B.false$	

$$S \rightarrow \mathbf{if} \ (B) \ S_1 \mid \mathbf{while} \ (B) \ S_1$$

SDD – II

Production	Semantic Rules
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow !B_1$	$B_1.true = B.false;$ $B_1.false = B.true;$ $B.code = B_1.code$

当跳转的时候是在内部的话就需要借助newlabel()生成一个标签辅助跳转



$S \rightarrow \mathbf{if} (B) S_1 \mid \mathbf{while} (B) S_1$

Example 6.22

```
if ( x<100 || x>200 && x!=y ) x = 0;
```

```
a<b  →  if a<b goto B.true
        goto B.false
```

```
a>b  →  if a>b goto B.true
        goto B.false
```

```
a!=b →  if a!=b goto B.true
        goto B.false
```

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel gen('if' E_1.addr \text{ rel } op E_2.addr 'goto' B.true)$ $\parallel gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$

Figure 6.37: Generating three-address code for booleans

```
if x < 100 goto L2
goto L3
L3:  if x > 200 goto L4
    goto L1
L4:  if x != y goto L2
    goto L1
L2:  x = 0
L1:
```

- Example 6.21 (参考)

```
if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2:  x = 0
L1:
```

Figure 6.34: Jumping code

8.1.2 目标程序

- 最常见的目标机体系结构是RISC(精简指令集计算机)、CISC(复杂指令集计算机)和基于堆栈的结构。
- RISC机通常有很多寄存器、三地址指令、简单的寻址方式和一个相对简单的指令集体系结构。相反，CISC机通常具有较少寄存器、两地址指令、多种寻址方式、多种类型的寄存器、可变长度的指令和具有副作用的指令。
- 在基于栈的机器中，运算是通过把运算分量压入一个栈，然后再对栈顶的运算分量进行运算而完成的。为了获得高性能，栈顶元素通常保存在寄存器中。因为堆栈组织的限制太多，并且需要太多的交换和拷贝操作，所以基于堆栈的机器几乎已经消失了。
- 但是，基于堆栈的体系结构随着Java虚拟机(JVM)的出现又复活了。JVM 是一个 Java 字节码的软件解释器。字节码是由Java编译器生成的一种中间语言。这个解释器提供了跨平台的软件兼容性。这是 Java 成功的一个重要因素。
- 解释执行会引起很高的性能损失，有时可能达到10倍的数量级。为了克服这个问题，人们创造了即时(Just-In-Time, JIT)Java编译器。这些即时编译器在运行时刻把字节码翻译成目标机上的本地硬件指令集。另一个提高Java程序性能的方法是建立一个编译器，把Java 程序直接编译成目标机器指令，彻底绕过字节码。

End of Chapter Six